

五階段操作細節(消融主線,以跨季標準化管線為準)

每一階段「實際怎麼做」——切折、訓什麼、怎麼調參、怎麼 blend、怎麼存檔。以跨季標準化 5 場(s4e1/s4e11/s5e10/s6e1/s6e2)的 04/05/06 腳本 + tree_search/ 為準; S3 早批做法相同、僅檔名探索式命名不同。以 s6e1(R^2 、630k 列)為貫穿範例。鐵則貫穿全程:所有階段共用同一組折索引與種子(seed=42),分數才可配對比較;一切決策看後處理後的分數,不看 raw;OOF 是唯一貨幣。

Contents

階段間差異一覽($N \rightarrow N+1$:只加一項能力)	2
階段 1:無 skill 基線(通用 AutoML 批次)	2
階段 2:+ kaggle-agent skill(結構化六階段流程)	3
2.1 特徵(features.py,確定性轉換)	3
2.2 CV	3
2.3 三模型,各自 5-fold OOF	3
2.4 集成:窮舉單純形網格權重搜尋	3
階段 3:+ 線性自我迭代(經驗庫先驗 + Optuna + seed bagging)	4
回合 1:經驗庫先驗的受控檢驗(信之前先驗)	4
回合 2:Optuna 調參(fold-0 代理目標)	4
回合 3:seed bagging(最便宜的殘餘增益)	5
集成:Dirichlet 隨機搜尋(取代網格,成員變多)	5
階段 4:+ 樹搜尋(ERA 第一支柱,取代線性單路徑)	5
4.1 狀態表示	5
4.2 節點評分(eval_blend,comp-agnostic)	5
4.3 展開 / 選擇 / 回溯	6
4.4 預算相位機 + 探索爆發	6
4.5 為什麼比線性強	6
階段 5:+ 外部想法注入 / 重組(ERA 第二支柱)	6
5.1 想法注入(plateau 觸發)	6
5.2 想法重組(LLM in the loop)	6
5.3 兩個關鍵誠實結果(這才是階段 5 的實質)	7
階段 6(交付面):產出提交前的重現閘門	7
附錄 A:同一套五階段骨架,如何隨「指標」變形	7
附錄 B:樹搜尋引擎演進 $v1 \rightarrow v2 \rightarrow v3$ (階段 4 內部)	8
一頁速查:各階段「加了什麼 + 怎麼做」	8

階段間差異一覽(N→N+1:只加一項能力)

消融的靈魂是「一次只動一個變因」。下表把每一步「唯一新增的能力」與「刻意保持不變的東西」分開列——差值才可歸因:

轉換	唯一新增能力	保持不變(控制變因)	典型效果
1 → 2	①手刻單腳本 FE → 模組化共用 <code>features.py</code> (非「有無 FE」之別:階段 1 已手打完整 FE)②第三成員換成 架構相異的 CAT (而非第二個 LGB)③早停(3000/100)④ 權重網格 0.1→ 0.05 ⑤ OOF 快取 + 位元級重現旗標	5 折、seed=42、GBDT 家族、加權平均集成	結構化 + 多樣性成員,穩定小升
2 → 3	自我迭代迴圈 :先驗受控檢驗 → Optuna 調最強單模 → seed bagging	同折、同特徵基底、成員池的既有 solo 全部沿用(加不替換)、集成仍是凸權重搜尋	中升;增益集中在「資料吃調參」的場
3 → 4	搜尋策略 :線性單路徑 → 候選樹 + 回溯 + 預算相位機 + mega-blend 爆發	同折、同評估器、同權重搜尋法、成員仍是 GBDT	場場正向或持平;贏家形狀就地找到
4 → 5	想法來源 :內部 → 外部注入(idea_bank)+ LLM 重組/提案	同折、同評估器、同搜尋引擎、同權重搜尋	重組=null(定理);注入接線後 2/5 顯著

關鍵洞見:1→4 每一步換的是「**怎麼找解**」(特徵→多樣性→調參→搜尋策略),成員池始終是同一批 GBDT;直到階段 5 才第一次試圖改變「**成員池的組成來源**」——這也是為什麼前四階段的增益受「同池凸最優」上限約束,而階段 5 的 LLM 擴池是唯一能碰那個上限的動作。

階段 1:無 skill 基線(通用 AutoML 批次)

目的:建立「一支腳本、手打一遍、無 skill 引導」的錨點。 **腳本**: `scripts/full_pipeline.py` (2 月通用批次,一支腳本跑完全流程)。 **操作**: - 特徵:**完整的手工特徵工程**(注意:不是「無 FE」),全部手刻在同一支腳本裡,六類——1. **序數編碼**: `sleep_quality/facility_rating/exam_difficulty` 用手寫字典 `.map()` → 0/1/2(保留順序); 2. **標籤編碼**: `gender/course/internet_access/study_method` 各一個 `LabelEncoder`; 3. **交互 + 平方**: `study×attendance`、`study×sleep_quality`、`study2`、`attendance2` ... 6 個; 4. **聚合分數**: `total_effort = (study/8 + attendance/100 + sleep_quality/2)/3`; 5. **方法效度分**: `{self-study:0...coaching:4}` 映射 × `study_hours`; 6. **睡眠特徵**: `sleep_deficit = 8 - sleep_hours`、`sleep×quality`。合併 `train+test` 後一起編碼(無目標洩漏);最後 drop 原始類別欄 + ID + target。 - CV: `cv_model_helper` 內 `KFold` (5 折)。 - 模型:**三個,但都在 GBDT 家族**——LGB、XGB、LGB_v2(第三個是「更多樹+更低 lr」的第二個 LGB,**不是獨立的 CAT**);各自手打超參、不調參、不早停(固定 `n_estimators`)。 - 集成:**網格權重搜尋,step=0.1**,在 OOF R^2 上找最佳(比階段 2 的 0.05 粗)。 - 產出:一份提交 + 分數;**不存 OOF 快取、無 deterministic 旗標、無共用折模組**。

階段 1 的「性格」(不是「陽春」,是「憑直覺、手刻、不可攜」):(a) 特徵全靠當下判斷、**無經驗庫背書**;(b) 全部寫死在單一腳本、無可重用模組(每場重寫一遍——這是「新競賽跑不動」的最底層來源);(c) 手打了一堆交

互/平方欄——而這個決定正是階段 3 第一回合用經驗庫先驗回頭檢驗的對象(「樹自己會學交互,顯式乘積只加共線」→對照 11 base 欄 vs 22 欄)。

與階段 2 的真實差異(增量歸因):不是「有沒有 FE」,而是①手刻單腳本 → 模組化共用 `features.py` ②第三成員換成**架構不同的 CAT**(而非第二個 LGB)③權重網格細化(0.1→0.05)④**OOF 快取 + 位元級重現旗標** ⑤統一實驗記錄。

階段 2: + kaggle-agent skill(結構化六階段流程)

目的:量「有結構化流程(EDA→CV→特徵→建模→評估→提交)+ 三模型加權」的增量。腳本: `scripts/04_train_blend.py` (+ 共用 `features.py`)。

2.1 特徵(features.py,確定性轉換)

- s6e1 為例:22 特徵 = 4 原始數值 + 3 序數編碼 + 4 標籤編碼類別(11 base)+ 11 工程欄(4 兩兩乘積、2 …)。
- 全部是確定性轉換,train/test 一致; `build_all()` 回 (`Xtr`, `Xte`, `FEATS`),各階段共用。

2.2 CV

- `make_folds(y) = KFold(5, shuffle=True, seed=42)` (連續 i.i.d. 目標,無時序/群組)。
- 折索引寫死、跨階段共用(這是配對比較的基礎)。

2.3 三模型,各自 5-fold OOF

三個「架構不同」的 GBDT,提供集成多樣性:

模型	預設超參(節錄)	迭代/早停
LGB	<code>lr=0.05, num_leaves=63, max_depth=-1, min_child_samples=30, feature_fraction=0.8, bagging_fraction=0.8, reg_alpha/lambda=0.1; deterministic=True, force_row_wise=True, num_threads=16</code> (位元級重現)	<code>num_boost_round=3000 + early_stopping(100)</code>
XGB	<code>lr=0.05, max_depth=7, subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1, reg_lambda=1.0, tree_method="hist"</code>	<code>n_estimators=3000 + early_stopping_rounds=100</code>
CAT	<code>lr=0.05, depth=8, l2_leaf_reg=3.0</code>	<code>iterations=3000 + early_stopping_rounds=100</code>

每個模型跑 5 折:每折用 4 折訓、對留出折預測 → 拼成 **OOF 向量**;對 test 則 5 折各預測後取平均(bagging)。
每個模型的 (`oof`, `pred`, `score`) 存進 `scripts/cache/solo_<NAME>.npz` → 階段 3/4 直接重用,不重訓。

2.4 集成:窮舉單純形網格權重搜尋

- 3 個成員 → 權重 (`w0, w1, w2`), `w0+w1+w2=1`、非負。

- **step=0.05 的網格窮舉**(雙迴圈掃 w_0 、 w_1 , w_2 由約束推出),每組在 **clip 後的 OOF R^2** 上計分,留最好的。
- 「直接對 clip 後分數搜」是鐵則:s6e1 目標 [0,100],所以 blend OOF 先 `clip(0,100)` 再算 R^2 。
- 產出:提交檔 `clip(test_blend, 0, 100)`、`blend_stage2.npz`、experiment log。

階段 3+: 線性自我迭代(經驗庫先驗 + Optuna + seed bagging)

目的:量「一條路走到底的自我改進」增量。 **腳本:** `scripts/05_iterate.py <round>`,分回合跑。

回合 1:經驗庫先驗的受控檢驗(信之前先驗)

- 鐵則:一條先驗必須先在本場指標上通過小型受控對照,才採信;否則丟棄並誠實記錄。
- 例(s6e1):
- 先驗 A「樹自己會學交互,顯式乘積欄只加共線」→ 對照「11 base 欄 vs 22 欄」同參同折,比 clip- R^2 ,贏者取得 pool 的 LGB 槽。
- 先驗 B「把預測 clip 回訓練值域」→ 零成本比 raw- R^2 vs clip- R^2 。
- 這一步把「經驗」變成「本場實證」,不是照單全收。

回合 2:Optuna 調參(fold-0 代理目標)

- 只調最強單模(高性價比槓桿);用 **fold-0 代理**控時間預算(全 5-fold 每 trial 會爆時)。
- 目標函式 = **fold-0 的 clip 後 R^2 直接優化**(對齊競賽指標,不調 RMSE 代理)。
- Search space(`suggest_*`):

超參	範圍	尺度
<code>num_leaves</code>	15-511	log
<code>max_depth</code>	4-14	—
<code>learning_rate</code>	0.01-0.15	log
<code>min_child_samples</code>	5-200	—
<code>feature_fraction</code>	0.5-1.0	—
<code>bagging_fraction</code>	0.5-1.0	—
<code>reg_alpha</code> , <code>reg_lambda</code>	1e-3-10	log

- `create_study(direction="maximize")`, `optimize(n_trials=40, timeout=1500)` (TPE 取樣器)。
- 優勝配置**重跑完整 5-fold**驗證,確認 fold-0 排序有遷移到全 OOF。
- 加入 **pool(不替換原成員)**:調參版當額外成員,原 LGB 留著,權重搜尋自行裁決(留著零成本、勝出與否由 OOF 決定)。這是「別替換多樣成員」教訓。

為什麼 fold-0 代理有效、又省時:全 5-fold 每 trial 要訓 5 次, $40 \text{ trials} \times 5 \approx 200$ 次訓練會爆時(甚至逾時整批歸零);改成「每 trial 只訓 fold-0 一次」→ 40 次訓練、s6e1 約幾百秒。前提是「fold-0 上的超參排序**會遷移**到

全 OOF」——這點在 s3e7/s3e1/s3e11 三場實證成立,所以優勝配置**只需重跑一次完整 5-fold** 確認,而非每個 trial 都跑。資料極小(~2k)時反而不需代理,全 CV 每 trial 才 ~1 秒(s3e5 即如此)。

「**加不替換**」的機制:調參版**不是**踢掉原 LGB,而是當**額外成員**塞進 pool,交給權重搜尋裁決。理由:調參版與原 LGB 在 OOF 空間高度相關,直接替換會流失多樣性、反而讓 blend 退步(s3e14 實測:替換 340.95 vs 增列 340.63)。留著零成本——沒用權重自然被壓到 0(s3e11 甚至把原 LGB 歸零、全交給調參家族)。

回合 3:seed bagging(最便宜的殘餘增益)

- 同超參、只換 seed=2024 再訓一個,當額外成員加入 pool(同樣加不替換)。
- 機制:同配置換種子 → 亂數起點不同 → 降「單一隨機起點」的運氣成分(降變異數),逼近天花板時常是唯一還能白拿的增益(s3e14 加 seed=2024 第 5 成員:340.627→340.599,成本一次訓練 ~52s)。
- **反例已記錄**:對「Optuna 直接優化最終指標」調出的模型,seed bag 可能無效(該解已收斂到淺而穩定、模型自身變異已低,第二個 seed 提供的多樣性有限)——權重被歸零(s3e5)。所以照樣「加入讓權重搜尋裁決」,不強推。這也是後來階段 5-A 機械版 seed bag 五場皆無增益的前兆。

集成:Dirichlet 隨機搜尋(取代網格,成員變多)

weight_search(): 1. 種子候選 = n 個單位向量 + 1 個均勻權重; 2. + rng.dirichlet(ones, size=800) (800 組隨機非負、和為 1 的權重); 3. 全部在 clip-R² 上計分,留粗選最佳; 4. **精修**:以粗選最佳 × 200 當 Dirichlet 濃度,再抽 300 組在其附近細搜。 - 每回合產出 blend_stage3_<tag>.npz + 提交 + log。

階段 4:+ 樹搜尋(ERA 第一支柱,取代線性單路徑)

目的:用「搜尋一棵候選樹」取代「一條路走到底」。 **程式**: tree_search/harness_v2/v3.py (引擎)、run_<comp>_v3.py (各場 driver)、eval_<comp>.py (各場評估器)。

4.1 狀態表示

- **節點** = 一個完整可評分解: {kind:"solo", model, params, features} 或 {kind:"blend", members:[node ids], weight_search}。
- **solo 節點**:實際重訓(5 折),OOF/test 存進 cache_<comp>/solo_<id>.npz。
- **blend 節點**:只讀成員快取 OOF、跑權重搜尋,不重訓 → 秒級。

4.2 節點評分(eval_blend ,comp-agnostic)

- 載入成員快取 OOF → 權重搜尋 → metric_fn(blended_oof) 計分。
- weight_search="dirichlet" :**k=1500** 組 Dirichlet + n 單位向量 + 均勻,再以最佳為中心細搜 k//3。
- weight_search="grid_simplex" :成員 ≤ 5 時窮舉 step=0.05 網格。
- **關鍵正確性**:任何後處理(取整/貼格點/OptimizedRounder 切點/clip)**必須寫進 metric_fn 內**,對每一組候選權重都套用——否則「raw OOF 上最好的權重」在後處理後未必最好。

4.3 展開 / 選擇 / 回溯

- **血脈(lineage)**:從根節點某個一代子節點衍生出的整條分支;每條血脈各自記「連續無改善次數 (streak)」。
- **選擇**(`select_next_parent`):在**非 plateau 的血脈**中,各取該血脈目前最佳節點,回傳其中分數最高者當下一個展開父節點(best-first,偏好高分血脈但仍分血脈探索)。
- **展開**:從父節點長變體(換超參 / 加成員 / 換特徵家族 / 併 blend);`dedup( config_hash 查重)`——重複配方直接擋、並告訴提議器「換一個」(避免浪費 plateau 預算在近重複上)。
- **回溯**(plateau):同一血脈**連續 3 個子節點(PLATEAU_STREAK=3)**沒改善 → 該血脈標記 plateau,選擇跳去別的血脈;**全部血脈都 plateau → 重開一次**(fallback,讓搜尋能跑到節點預算)。
- v2 進階:plateau 門檻「metric-aware / adaptive」(依 tie_rate 調整)。

4.4 預算相位機 + 探索爆發

- `init_budget`:預設 **總 60 節點 / burst 6 / patience 20**,分「先廣後深」相位。
- 相位翻到 `explore_burst` 時,強制注入 **mega-blend**(把**全部成員**混成一個 kitchen-sink blend)+ 長射程種子——歷史上很多增益來自這個強制大混。
- 停止準則:全域耐心用盡即停(記 `stop_reason`)。

4.5 為什麼比線性強

不同場「贏家形狀」不同(有的 mega-blend 奪冠、有的血統 blend、有的單模最好);搜尋**就地找到對的形狀**,不必事先賭。s3e16 用樹搜尋最佳解實際提交,Public/Private 雙榜都比只用 skill 好(本地優勢非過擬)。

階段 5:+ 外部想法注入 / 重組(ERA 第二支柱)

目的:內部想法用盡後,從外部(論文/write-up/教科書)引入新想法,或把兩個高分解交 LLM 重組。**程式**: `idea_injection.py` (注入機制)、`prior_wiring.py` (接線翻譯器)、`harness_v4.py` (重組算子)。

5.1 想法注入(plateau 觸發)

- 當階段 4 plateau、推不動時,把「與本場相符、且**還沒試過**」的 [EXT] `idea_bank` 條目 **翻譯成搜尋能評估的候選 config**,丟進搜尋。
- 每次注入至多 K 條、每場至多 M 次;候選標 [EXT-NN] 供歸因。
- `comp-agnostic:translator` 吃 (`ext` 條目, 目前最佳解成員, `ctx`) → 回候選 config;新增想法只要在 `TRANSLATORS` 加一條。

5.2 想法重組(LLM in the loop)

- 讓 LLM 比較兩個高分節點的核心概念、合成新候選(本專案第一個「LLM 在搜尋迴圈裡」的動作)。
- **propose-once/freeze/replay**:LLM 提案一次、凍結成 config,之後評估重放這份檔 → 可重現。

5.3 兩個關鍵誠實結果(這才是階段 5 的實質)

1. **重組 = honest null(有機制證明)**: 樹搜尋冠軍 = 成員池的 **NNLS 凸最優**(逐位相符), 成員誤差相關 0.99+; 固定池內任何重加權 $\leq$ 凸最優 $\rightarrow$ 重組不可能贏(見 `recombine_findings`)。
2. **注入曾是 write-only bug**: 先驗撈出後存進 `tree['priors']`、**搜尋端從未讀**(見 `bug_prior_injection_noop`)。2026-07-13 接線後做三臂 $\times$ 5 場對照:
3. **機械版(A)**: 規則模板把先驗翻成候選, 決定性可重現; 5/5 冠軍不變(驗證凸最優定理)。
4. **LLM 版(B)**: LLM 讀先驗+戰況即時提案、注入**去相關新成員**擴池; 2/5 場統計顯著(s6e1 R^2 、s4e1 AUC), 量級 $1e-5$, 價值在機制歸因(見 `prior_wiring_findings`)。

階段 6(交付面): 產出提交前的重現閘門

腳本: `06_rebuild_tree_best.py`。- 讀樹取冠軍節點 $\rightarrow$ 把每個成員**從頭重訓** $\rightarrow$ 逐位比對「新 OOF」與「搜尋時快取 OOF」: - GATE 1: 分數逐位相符; GATE 2: 恢復的權重 `round(4)` 逐成員等於樹上存值。- 兩閘門過才放行產提交檔; 抓「靜默污染提交檔」的 bug。- 依賴 LGBM `deterministic/force_row_wise/num_threads` 固定達位元級重現(否則跨程序不可重現)。

附錄 A: 同一套五階段骨架, 如何隨「指標」變形

五階段的**流程骨架完全相同**, 但兩個地方**必須隨競賽指標換零件**——**CV 切法**(隨資料結構)與**後處理**(隨指標的貝氏最優解)。這是「好的驗證 + 針對指標優化」鐵則的具體落地:

場	指標	方向	CV 切法	後處理(寫進 <code>metric_fn</code> , 對每組權重都套)
s6e1	R^2	max	<code>KFold(5, shuffle)</code>	<code>clip[0, 100]</code> 後算 R^2
s5e10	RMSE	min	<code>KFold(5, shuffle)</code>	<code>clip[0, 1]</code> 後算 RMSE
s4e1	ROC-AUC	max	<code>StratifiedKFold</code> (分類)	無(AUC 只看排序, 對單調變換不變)
s4e11	Accuracy	max	<code>StratifiedKFold</code>	門檻重擬 (在 OOF 上搜最佳 cutoff 再算命中率)
s3e5	QWK	max	<code>StratifiedKFold</code> (序數分層)	OptimizedRounder : 在 OOF 上搜 K-1 個切點把連續預測切成等級
s3e16	取整 MAE	min	分箱 <code>StratifiedKFold</code>	四捨五入到整數 + <code>clip</code>
s3e20	時序迴歸	—	LOYO (留一年, 時間外推)	<code>clip</code>

要點: 1. **後處理一定寫進評估器的 `metric_fn` 內**, 不是事後補在冠軍身上——否則「raw OOF 上最好的權重」在後處理後未必最好(離散指標尤其致命)。這使樹搜尋對 QWK/accuracy 這類「切點/門檻」指標也天生正確。2. **符號統一**: harness 內部一律「**越小越好**」; maximize 指標(R^2 /AUC/QWK/accuracy)一律以 `-metric` 餵進去, `result` 裡再存回人類可讀的正值。3. **CV 隨資料結構, 不隨指標**: 分類用分層、時序用 LOYO、有重複實體用 `GroupKFold`——這一層與階段無關, 五階段共用同一組折。

附錄 B:樹搜尋引擎演進 v1 → v2 → v3(階段 4 內部)

階段 4 的引擎本身也是迭代出來的,每版只加「被證據要求的東西」:

v1(harness.py) — 骨架:節點/血脈、`select_next_parent`、`global_best`、`PLATEAU_STREAK=3` 回溯、全血脈 plateau 時「重開一次」fallback。

v2(harness_v2.py) — 加三件: - **通用 blend 評估器** `eval_blend` (comp-agnostic:Dirichlet k=1500 + 精修,後處理走 `metric_fn`); - **config 去重**(`config_hash` ;近重複配方直接擋,並要求提議器換一個); - **metric-aware / adaptive plateau**(依 `tie_rate` 調 `streak` 門檻)+ 經驗庫 `mutation prior`。

v3(harness_v3.py) — 加「預算與收尾政策」六件(全來自 `scaling` 實驗證據): 1. **預算相位機**(`init_budget` ,預設總 60 節點): `exploit` → `explore_burst` → `stopped` ——所有血脈 plateau 才進 `burst`; 2. **強制 explore_burst**(min 5、預設 6 evals 窗):driver 注入 5-8 條長射程血脈 + **mega-blend**;burst 內 15-20 評估沒改善就停;證據:`scaling` 曲線上「exploit 相後的 3 次全域最佳改善,全部來自強制 burst」; 3. **dedup 消耗預算**(擋重複時消耗一個展開槽,避免空轉); 4. **solo 突破自動重開 blend 血脈**(新 solo 成為全域最佳時,已 plateau 的 blend 血脈解凍去吸收它——修「深層 CAT solo 在 blend 血脈 plateau 後才出現、要人工重開」的坑); 5. **完整搜尋狀態持久化**(`phase/flags/id-name` 對映可存取還原,支援中斷續跑); 6. **burst 種子健全性閘門**(長射程種子要先過基本檢查才准衍生子節點)。

跨季 5 場用 v3;S3 早批多為 v2 / v1(這也是接線實驗只挑跨季場的原因——v3 標準才有 `search_state.node_results` 存真權重、逐位重現)。

一頁速查:各階段「加了什麼 + 怎麼做」

階段	新增能力	CV	調參	集成	存檔
1	無 skill(手打一遍)	KFold5	手打超參	網格 step0.1 (3×GBDT,含 2 個 LGB)	只出提交
2	結構化流程 + 3 模型	KFold5 共用折	預設超參	網格 step0.05 (clip-R ²)	<code>solo_*.npz</code>
3	先驗檢驗 + Optuna + seed bag	同折	fold-0 代理 Optuna 40 trials (直接優化指標)	Dirichlet 800+精修300	<code>blend_stage3_*.npz</code>
4	樹搜尋(候選樹/回溯/相位機)	同折	搜尋內展開	每 blend 節點 Dirichlet k=1500 ;mega-blend 爆發	<code>cache_<comp>/solo_<id>.npz</code>
5	外部注入 + LLM 重組	同折	LLM 提案凍結重放	擴池後重 blend;NNLS 驗證	<code>wire/ledger.json</code>

貫穿鐵則:同折同種子(配對比較) • 只看後處理後分數 • OOF 是貨幣 • 加成員不替換 • 一切以逐位重現閘門
收尾。